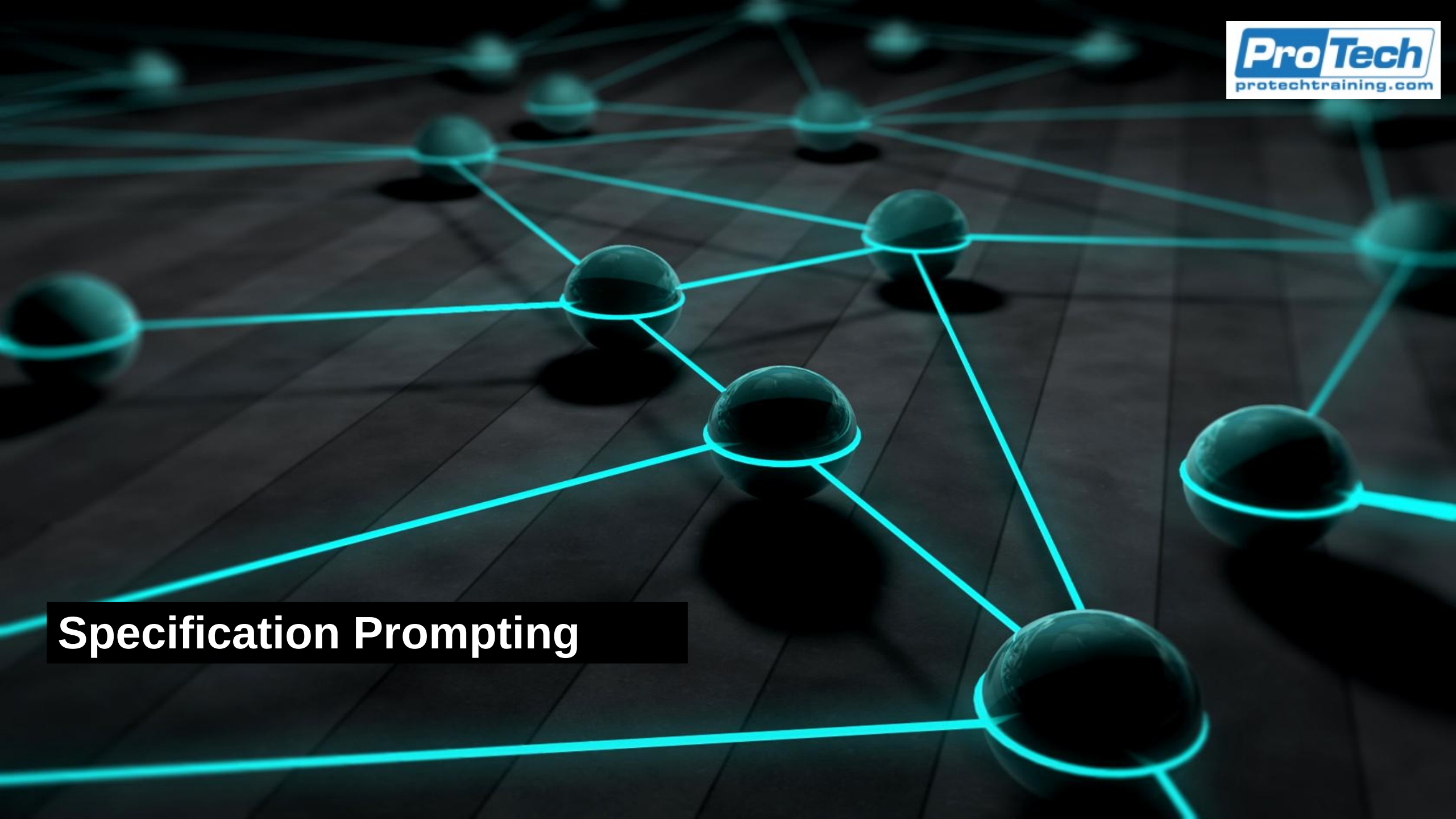


Specification Prompting

Specifications

- A precise description of what a system feature must do
  - A specification answers the question
    - What must this feature do for it to meet the stakeholder requirements?
    - Specifications are developed before anyone tries to build the feature
    - Provides a design target and scope and forces an analysis of that is to be done before we start working
  - The civil engineer planning a bridge specifies
    - The load it must carry, the span, the wind tolerance, the material strengths required
  - The pharmaceutical chemist researching a drug formulation specifies
    - Active ingredient mass, pH range, dissolution rate, and shelf-life conditions
  - Specifications do not describe how to build anything
  - Specifications are describes the acceptance criteria
    - The describe how the final product must look and act in order to meet the stakeholder requirements
    - Standard engineering practice, but it does assume we understand the requirements

Requirements

- A requirement is a statement of what a stakeholder needs or wants
  - “I need to be able to cross this river without using a boat”
  - “I need to be able to find all of our customers that live in a specific city.”
- A statement is a requirement if you can word it like this
  - “I need this, and I don’t care how you do it for me”
  - A requirement does specify a feature, that is the responsibility of the developers
  - Every specification describes a feature to be build
    - The specification defines how a feature looks like and how it behaves
  - Each of the features satisfies one or more requirements directly or indirectly
    - Ever feature specified should trace back to one or more requirements
- The description of how to build something from a specification is a design
  - Designs depend on the resources available to build what is specified

Role of Specifications

- Writing a specification allows us to catch design errors before construction
  - We have seen how the cost of fixing errors increases the later in the development and deployment process we find it.
- Allows multiple teams to work in parallel
  - For example, the front end and back end teams can work on different parts of the specification
- Provide acceptance criteria for testing
  - We can’t write tests until we know what the product is supposed to do
  - Specifications are often referred to as a testing and quality baseline
- Creates a permanent record for maintenance
  - Specifications outlive the original authors
    - For example, five years after a system ships, the spec may be the only document that explains why it does what it does.

IEEE 830 and Its Successor

- IEEE 830-1998
  - “Best Practices for Software Requirements Specifications”
  - Defines what a Software Requirements Specification document should contain and what makes one well-formed
    - It was superseded in 2011 by ISO/IEC/IEEE 29148, which folded the same content into a broader requirements-engineering lifecycle standard
    - The quality attributes IEEE 830 listed are unchanged in the newer standard, which is why people still refer to "IEEE 830 quality" in practice
  - One of the most cited references in software engineering
  - Aligns software specifications with the best practices in other area of engineering

Quality Characteristics

- Apply to any type of specification, not just formal documents
  - Correct
  - Unambiguous
  - Complete
  - Consistent
  - Ranked
  - Verifiable
  - Modifiable
  - Traceable

Correct

- Correct
  - Every specification is consistent with how the business works
  - The software follows the business procedures and rules
    - For example, we should not be able to withdraw negative money from a bank account because that is not the way back accounts work in the real world
  - Also concerned with whether the specifications are the right ones for the project
    - For example, specifications about financial regulatory compliance for a system that has nothing to do with financial operations or systems
  - Helps identify
    - Whether specifications have been added to the project from some standardized set of requirements, but they don’t apply to this project
    - Whether specifications have been added, often cut and pasted from similar projects, without an adequate review process
    - Specifications that should be part of the project but are somehow missing
    - For example, resilience specifications for a mission critical system are missing

Unambiguous

- Unambiguous
  - Each requirement has exactly one interpretation
  - Every reader gets the same understanding as the original author
  - People who write the spec often are blind to other interpretations
    - Which is the same reason we can’t proofread our own writing
    - We know what we meant so we are blind to other interpretations
  - Standard approach to ensure we avoid ambiguity is called a spec review
    - Domain experts read through a specification specifically to look for ambiguities
    - Important they were not involved in writing the spec
    - The serve as a “fresh set of eyes”
  - Basic best practice
    - A specification is not finalized unless it has been reviewed by independent and impartial domain experts

Complete

- Completeness
  - Identifies how the system responds to every type of input in every situation
    - Including invalid and unexpected input, in addition to expected and valid input
  - Ensures we have not overlooked valid inputs
    - Often happens because some very statistically rare outliers are valid but unexpected
  - Ensures we know how the system should respond
    - When invalid input or unexpected input is encountered
    - Describes what classes of input should be rejected to avoid system errors
    - And describes how to fail safely instead of crashing
  - Forces a focus on the edge cases the system might encounter
    - This can be very difficult to do for inputs that are statistically very rare, both valid and invalid
  - The goal is a resilient and robust system that can handle all possible inputs
    - That means avoiding unstable system states that can attackers could exploit
    - At the very least, the system shuts down gracefully rather than just crashing

Consistent

- Consistent
  - There are no internal contradictions or conflicts with other specs
  - Correctness is about consistency between the system and the business
  - Consistency is about the internal consistency among the specification items
  - For example, an inconsistent spec might have these items
    - A section that says "passwords must be 8 to 14 characters in length"
    - And "the password field accepts up to 16 characters" in another section
    - A 15 character password is not allowed by the first spec item but allowed by the second spec item
  - The problem is that a developer can't do both and will chose one or the other
    - This eventually results in potentially
  - Often, the underlying problem is often that two different groups of stakeholders have conflicting requirements but they are both included.
    - The result is an inconsistent spec

Ranked

- Ranked
  - Provides a way of identifying priorities among the spec items
    - If there is not enough time to do it all, ranking ensures the most critical or important things get done first
  - Normally a collaborative effort
    - Business side identifies what is most important to the business
    - Development side identifies the time, cost and effort required to develop features
    - The resulting ranking takes both of these into consideration
  - Criteria for ranking a project dependent
    - May be risk based, usage based or any other criterua that is important to the project
  - Common ranking
    - High, medium, low
    - Must, Should, Could, Won't

Verifiable

- Verifiable
  - For each specification item, it must be possible to write a pass or fail test
  - Means that we need to quantify what the acceptable output is
    - “The system should only allow secure passwords” is not verifiable
    - The problem is that we have no idea what is meant be “secure”
    - Passwords should be between 16 and 32 character and should include at least one number, one special character and one capital letter” is verifiable since we can write pass/fail tests to this statement
  - Forces all stakeholders to accurately define features with measurable definitions
    - Otherwise developers have no idea if they are satisfying the specification
    - Removes the chance of developers guessing at what the specification means
  - If you can’t figure out a test case for a spec item because you aren’t sure what it should do
    - Then you don’t know enough to write the code for that same spec item
  - Also includes the caveat that for each test
    - There is a finite and cost effective way of running the test

Modifiable

- Modifiable
  - Means the spec is organized so a single requirement can change without rewriting the whole document
    - Specifications numbered and referenced by ID are modifiable
    - Requirements buried in monolithic blocks of formal prose are not
  - Modifiable is often associated with clarity
    - Specifications that use diagrams, screenshots, other media and direct language to be precise
    - Preferred over specifications that are massive volumes of formal sounding text
  - It is considered a best practice of making modifiable specs is to use versioning
    - Putting specs under version control allows the team to track changes to the specification over time
    - Also ensures that the versions of the code and other deliverables are pinned to the spec they were developed to

Traceability

- Traceable means
  - Each requirement can be linked backward to where it was formulated
    - A customer interview, a regulation, a business goal
  - Each specification item can be traced:
    - Back to a requirement
    - Forward to the design elements, code modules, and tests that implement it
  - Traceability is mandatory in regulated industries
    - Medical devices, aviation, automotive, banking, etc
    - However it is good practice everywhere.

Validity

- Validity
  - Often included in addition to the others
    - Not part of the IEEE standard
  - For all everyone involved in the project agrees that the specification
    - Is reasonable and realistic
    - Describes a project that can be done successfully
    - Satisfies the most important requirements for the project
  - Often considered a reality check before work beings

Why Specifications Matter

- Production software has a long lifetime
  - A spec is not just for the first build; it is for the next ten years of changes.
- Multiple developers work on the same code over years
  - Software in production is typically maintained by people who were not on the original team
  - Without a spec, every change requires reverse-engineering intent from the code.
- Failures have real cost
  - Specifications are guides to avoid failures, especially when engaged in upgraded existing production systems
- Audit and compliance often require specifications
  - In regulated industries like finance and healthcare, specifications are not optional
    - They are required artifacts for certification, audit, and post-incident review

Specifications as Communication

- Spec is the contract between product, engineering, and QA
  - Serves to integrate these groups with a product or project focus
    - Product managers, who use it to confirm the feature matches the business goal
    - Engineers, who use it to design and build
    - QA, who use it to write tests.
  - Resolves disagreements before they become rework
- Onboards new team members faster than reading code
  - New team members typically come up to speed faster by reading specs than by reading cod
  - Because the spec carries intent; code carries only behavior.
- Documents the "why" behind the "what"
  - The "why" matters more than people initially expect
  - Six months in, no one remembers why a particular feature exists, and without a documented reason, future engineers may remove it.

AI Assistants Need Specifications

- The model has no prior knowledge of your project
  - Each session starts effectively from zero
  - The model cannot ask questions about strategic intent
  - Underspecified requests produce plausible but wrong output
  - An AI assistant does not learn over time
    - Treats every request as if seeing your project for the first time
    - It does not know your naming conventions, your architectural preferences, your error-handling style, your team's vocabulary for domain concepts, or which libraries you have decided to standardize on
    - It can ask clarifying questions if you set it up that way, but those questions are limited and will not reliably surface every gap
  - The result of an underspecified request is code that looks reasonable but does not fit your codebase.
  - The better the spec given to the AI, the better the result it produces

AI Assistants Need Specifications

- What an AI assistant sees
  - Your prompt text
  - Open files in the editor (with Copilot Chat in VS Code)
  - The repository if granted access
  - Configured instruction files (covered next)
  - Nothing else, does not see:
    - Decision made at the last team meeting
    - The project design docs on Confluence
    - The team's Slack thread about why you decided not to use Spring Data, the regulatory constraints your industry imposes, or any other context that is not in one of the items above
  - The most reliable way to communicate with the AI tool is configured instruction files
  - Because they persist across sessions and across team members.

AI Assistants Need Specifications

- Default behaviors
  - Pattern matches to similar code in training data
  - Picks popular but possibly outdated libraries
  - Uses generic naming and error handling
  - Invents plausible APIs that do not exist (hallucination)
- Caused by giving an AI assistant an underspecified request
  - It has to fill in the gaps somehow
    - By default it pattern-matches against what is most common in its training data
    - If your project uses something different, you will get inconsistent code
  - The model can also confidently produce method calls, library functions, or API endpoints that do not exist
    - This is called hallucination and is the highest-cost failure mode.
    - A specification that names exact dependencies, signatures, and constraints reduces hallucination dramatically.

AI Assistants Need Specifications

- A spec for an AI assistant is also a spec for a person
  - Everything you have learned about good specifications for human readers applies to AI assistants
    - The IEEE quality attributes work when providing a spec to an AI assistant
    - The discipline of writing verifiable, unambiguous specification produces higher quality results
  - The addition is that an AI assistant needs is an explicit statement of the context that a human team member would absorb implicitly
    - A senior engineer on the team knows you prefer constructor injection over field injection because they have been in the design reviews
    - The AI does not, so you state it explicitly
  - This is not a different skill from human-facing spec writing
  - It is the same skill applied with one more reader in mind who needs *everything* explained

Spec Channels for Copilot

- Inline prompts in chat
  - Copilot in VS Code reads from several channels in addition to the chat prompt.
    - Custom instructions: .github/copilot-instructions.md
      - The repository-wide instructions fileincluded automatically in every chat in that repo.
    - Prompt files: .github/prompts/*.prompt.md
      - Prompt files are reusable, invocable as slash commands
    - Custom chat modes: .github/chatmodes/*.chatmode.md
  - Spec Kit: structured spec-driven workflow
    - Official toolkit for spec-driven development
    - Provides templates, commands, and workflows
    - The awesome-copilot repository on GitHub has community-contributed examples of each of these.

A Good Copilot Spec

- Minimum five elements
  - Goal: what is being built and why
    - Anchors the rest because answers "why are we doing this"
  - Scope: explicit in-scope and out-of-scope items
    - Draws boundaries
    - Explicitly saying what is out of scope prevents the assistant from over-helping.
  - Interfaces: function signatures, endpoints, schema
    - Most important section for code generation
    - The more precise the function signatures, endpoint URLs, and data schemas, the more reliable the output.
  - Constraints: stack, libraries, versions, conventions
    - Prevents the assistant from picking a different library or framework than the one you have standardized on
  - Acceptance: how success is verified
    - How you and the assistant will both know the task is done
    - Ideally it is a list of test cases.

Examples

- Bad spec
  - "Write a function to calculate tax"
    - Underspecified on: language, signature, currency, rounding, error handling, jurisdictions, return type
    - Result: plausible code that probably does not fit the project
- Good spec
  - Comprises
    - Write a Java method calculateSalesTax(BigDecimal amount, String stateCode)
    - Returns BigDecimal with 2 decimal places, rounded HALF_UP
    - Uses rates from injected TaxRateProvider
    - Throws UnknownStateException for invalid state codes
    - Style: Spring Boot 3, Lombok allowed, follow project ESLint config
  - Even better is to put the framework-level constraints into .github/copilot-instructions.md
    - Then they are not repeated them in every prompt
    - The per-task prompt then only carries what is unique to that task.

Examples

- Bad testing spec
  - "Write tests for this function"
    - Tends to produce tests that pass trivially
- Good test spec
  - Removes any chance for the AI assistant to “make a good guess”
  - For example, it should specify details like:
    - Generate pytest tests for calculate_sales_tax
    - Cases: 3 known states (happy), 0 amount, negative amount, invalid state code
    - Use pytest.mark.parametrize for the 3 happy cases
    - Test naming: test_<scenario>_<expected_result>
    - Each test must have an Arrange-Act-Assert structure
  - The case enumeration covers the four IEEE-style completeness categories
    - Happy path, boundary, invalid input, and error path.

Examples

- Bad security spec
  - “Make this secure”
    - Tends to produce tests that pass trivially
- Good security spec
  - Specifies quantifiable specifics like
    - Review this Flask endpoint for OWASP Top 10 categories
    - Specific focus: A01 (Broken Access Control), A03 (Injection)
    - Report each finding with: severity, location, CWE ID, fix
    - Suggest test cases that would have caught each finding
    - Do not modify the code
  - The last instruction, "do not modify the code", keeps the assistant in analysis mode rather than starting to refactor.

Examples

- Bad documentation spec
  - “Document the code”
    - Usually results is a massive spew of rambling text
    - Difficult to read and often just states the obvious
- Good security spec
  - Uses factors like
    - State the audience (API consumer, internal developer, end user)
    - Specify the format (Markdown, OpenAPI, Javadoc, ADR)
    - List the sections required
    - Name the tone and reading level

Examples

- Documentation requests fail when the audience is unclear
  - An API reference for external consumers, an architecture decision record for the team, a Javadoc on a private helper, and an end-user guide are all "documentation" but have nothing else in common
    - Specifying audience and format constrains the output
    - Listing the required sections (overview, parameters, return value, examples, error codes, version history) ensures completeness
  - Naming the tone is sometimes necessary
    - The same content reads differently in formal Javadoc style than in a friendly README.

copilot-instructions

- The .github/copilot-instructions.md file
  - One file at the repository root path
    - Read by Copilot in every chat in that repo and contains the constraints common to all work in the repo
    - Versioned with the code
  - Read on every interaction
    - Anything you put in it propagates to every team member's Copilot automatically
  - Good content for this file
    - The tech stack and versions
    - The project's architectural rules
    - The testing framework
    - The naming conventions, the security and logging rules
    - The directories Copilot should and should not modify
  - Bad content
    - Anything specific to a single feature
    - Put that in a prompt file or in the chat prompt itself
  - The file is versioned, so changes go through pull request review like any other code change.

copilot-instructions

- Prompt files and chat modes
  - Prompt files: reusable task templates, invoked as slash commands
  - Chat modes: persistent personas with their own context
  - Both live under .github/
    - Both versioned with the code
    - Both contribute to consistency across the team
  - Prompt files solve the problem of writing the same long prompt over and over
    - A generate-unit-tests.prompt.md file under .github/prompts/ can be invoked as /generate-unit-tests in Copilot Chat
  - A chat mode is a saved configuration that includes instructions, allowed tools, and persona description
    - A "database administrator" chat mode might have read-only access to schema files and instructions on how to propose migrations
  - The point is that all of these are versioned with the code, so the team's conventions for using the AI assistant are documented code, not lore that people “just pick up.”

Best Practice

- Write the spec before you prompt
  - Outline goal, scope, interfaces, constraints, acceptance first
  - Paste the outline into the prompt
  - The discipline of writing it improves the prompt
  - Reusable outlines become prompt files
- Separates AI-assisted productivity from AI-assisted churn
  - Five minutes spent writing a spec can save twenty minutes of rework on the generated code
  - If you cannot articulate what you want, the assistant cannot guess it.
  - As an additional benefit, the discipline of writing the spec exposes ambiguities in your own thinking; many "tricky" coding tasks become straightforward once you have to specify exactly what they should do.

Best Practice

- Name the failure modes
  - Specify what happens when input is invalid and when external services fail
  - Specify what happens at boundaries (empty, max, negative, null)
  - These are usually where AI-generated code falls down
  - The happy path is what AI assistants do well by default
    - The interesting work, and the buggy work, is around the edges
  - A spec that says
    - "Raise ValueError for invalid input, return the empty list for zero results, retry once with exponential backoff for transient external errors, log and fail closed for unrecoverable errors"
    - Gives the assistant the framework to produce robust code.
    - A spec that does not mention these cases will produce code that ignores them
  - This is also where review effort is best spent
    - If the spec covers the failure modes, the review confirms the implementation matches; if the spec does not, the reviewer has to recommend the appropriate adjustments

Best Practice

- Ground in real artifacts
  - Reference
    - An existing file as the style template
    - A real schema or OpenAPI spec
    - Test fixtures by path
  - Concrete examples are more reliable than abstract descriptions
  - When you say "follow the style of OrderService.java" and attach that file to the chat
    - The assistant has a concrete pattern to match
    - This is more reliable than describing the style in prose
    - The same principle applies to data; pasting a real JSON sample is better than describing the schema in words.
  - VS Code's Copilot Chat supports attaching files directly with the paperclip icon or by typing # and selecting a file.
  - This should be used this aggressively.

Common Error

- Vague verbs
  - "Make this faster" or "Improve the code" or "Refactor for readability"
  - All produce changes; none produce predictable changes
  - Verbs like "make", "improve", "optimize", "refactor", and "clean up" without further specification
    - Are the most common cause of AI output that does the wrong thing
  - "Make this faster"
    - Might produce code that is faster on the hot path but slower on the cold path
    - Or adds a cache with unbounded memory growth
  - "Refactor for readability"
    - Might pull out functions that obscure the logic or add comments that just restate the code
  - Replace these verbs with measurable goals
    - "Reduce the loop in process_orders from O(n^2) to O(n log n) without changing the public signature",
    - "Extract the validation logic from submit_form into a class named FormValidator".

Common Error

- Scope creep in prompts
  - One prompt requesting many unrelated changes
    - Assistant does some well, others poorly
    - Hard to review the diff
    - Hard to roll back a single mistake
  - A prompt that says "add input validation, fix the off-by-one error, add logging, update the tests, and refactor the helper class"
    - Will produce a mix of good and bad changes that are difficult to untangle.
  - Smaller prompts produce smaller diffs that are easier to review and revert
    - The natural unit is one logical change per prompt; this also matches the convention for one logical change per commit
  - Many engineers find that breaking down a task into a sequence of prompts is the same activity as designing the implementation
    - The prompt sequence becomes the implementation plan.

Common Error

- Trusting the assistant for security and correctness
  - Generated code may compile and pass tests but be wrong
  - Generated security advice may miss real vulnerabilities
  - Always review; treat AI output like junior-developer output
- Specs make review faster, not optional
  - Code that compiles and passes the tests you wrote is not necessarily correct
    - The tests reflect what you thought to check.
    - AI-generated code is competent but not authoritative
    - It should go through the same review process as code from any other source
  - The spec is what makes review feasible at speed and scale
    - With a clear spec, the reviewer can check the diff against the spec in minutes
    - Without a spec, the reviewer has to derive intent from the code, which takes much longer.
  - Security advice from AI assistants is most useful as a checklist for the human reviewer, not as the final word

Common Error

- Not using the instructions file
  - Team agrees on conventions in meetings, never writes them down
  - Each engineer prompts in their own style
  - Generated code is inconsistent across the codebase
- The fix is one file: .github/copilot-instructions.md
  - The pattern is recognizable
    - A team adopts Copilot, productivity goes up briefly
    - Then the codebase fills with inconsistent code because each engineer's prompts encode slightly different defaults
  - A team's conventions are an asset
    - Codifying them in a single instructions file means they get applied to every new piece of code without anyone having to remember to mention them
    - Updating that file is usually a low-traffic activity; once stable, it changes only when the team's conventions change.

Version Control

- Specifications are code-like artifacts
  - They change as understanding evolves
  - They have authors, reviewers, and history
  - They belong in the same VCS as the code
- A spec that is not under version control is not a spec but a snapshot
  - Without history, you cannot answer
    - "When did we decide X"
    - "Why did we change from Y to Z".
  - Without review, you have no quality gate
  - Without branching, you cannot evolve the spec on the same branch as the code that implements the change.
  - Use the same repository, the same branching strategy, the same pull request review for the spec as for the code

Version Control

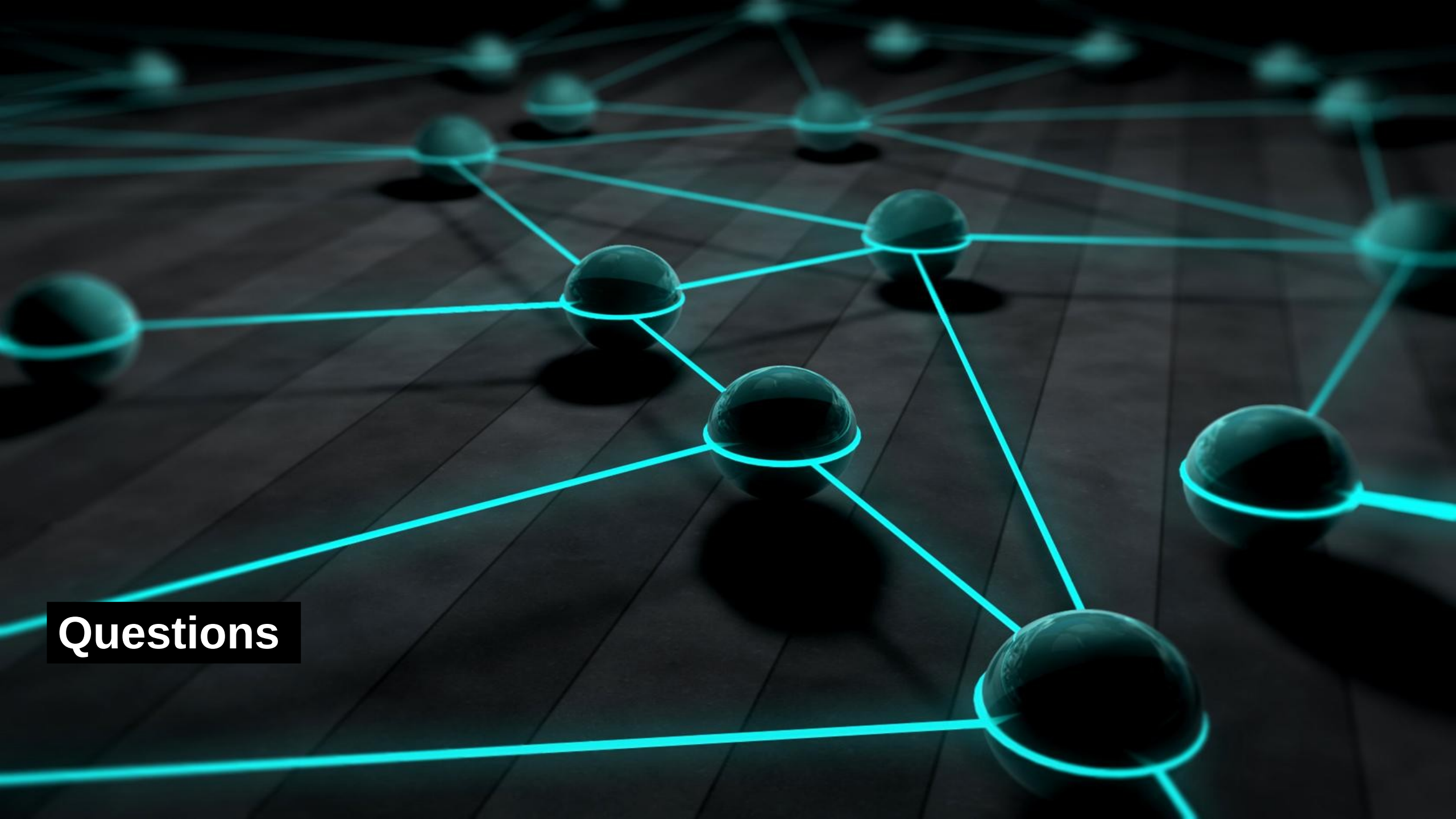
- Where specifications live
  - README.md: project overview and getting-started
  - docs/: longer specs and design documents
  - .github/copilot-instructions.md: AI-readable conventions
  - .github/prompts/: reusable task specs
  - ADRs in docs/adr/: significant design decisions
  - This is a sensible default layout for a project
    - The README is the entry point and a high-level spec for the project as a whole
    - The docs/ folder holds longer specs that are too detailed for the README
    - The .github/ folder holds AI-readable artifacts, all versioned and reviewed.
    - ADR stands for Architecture Decision Record; each ADR is a short document explaining one significant design decision, when it was made, what alternatives were considered, and why this one was chosen
    - ADRs are particularly valuable because they capture intent that would otherwise live only in people's heads.

Specs and Pull Requests

- Spec changes go through pull request review
  - Spec changes that affect AI output are reviewed for that effect
  - A failed implementation may indicate a spec defect, not a code defect
  - Treat the spec as the source of truth
- A pull request that updates .github/copilot-instructions.md should be reviewed with the same care as a pull request that updates the code
  - Changes to the instructions affect every future generation, so the blast radius is large.
  - When generated code does not work
    - The first question is "does the spec correctly describe what we want"
    - If it does, the question is "why did the assistant not follow it"
    - If it does not, fix the spec first and regenerate
  - Spec-driven development inverts the traditional flow
    - Code is regenerated from the spec, not patched by hand.

GitHub Spec Kit

- Official GitHub toolkit for spec-driven development
  - Open source: github/spec-kit on GitHub
  - Provides templates, slash commands, and a structured workflow
  - Works with Copilot, Claude Code, and other agents
- Spec Kit is GitHub's official open-source toolkit
  - Used for operationalizing what this module has described
    - It installs into a repository, adds prompt files and templates under .github/, and exposes slash commands like /speckit.specify, /speckit.plan, and /speckit.implement
    - Each command corresponds to a phase of the spec-driven workflow.
    - It is not the only way to do spec-driven development, but it is a concrete reference implementation worth exploring
    - The awesome-copilot repository, also on GitHub, has community-contributed templates that complement Spec Kit.


Questions